

flexible-attendance-app

開発環境の土台作り

GitHub Codespaces での開発開始ガイド

まずはリポジトリを作成し、Codespaceを立ち上げるところからスタートします。

リポジトリの準備と起動

1. GitHub上で新しいリポジトリを作成 します。
(今回は名前を `flexible-attendance-app` としている)。
2. 「Code」 ボタンをクリックし、「Codespaces」 タブから 「Create codespace on main」 を選択して起動します。

プロジェクト構造の初期化（推奨）

Spring Boot（バックエンド）と Vue.js（フロントエンド）を同じリポジトリで管理する場合、以下のようなディレクトリ構成にすると管理がしやすいです。

```
/flexible-attendance-app
├── backend (Spring Bootプロジェクト)
├── frontend (Vue.jsプロジェクト)
└── docs (myapp_structures.md などのファイル)
```

最初に行う環境構築コマンド

Codespacesのターミナルで、以下のツールのインストール有無を確認しましょう。

- Java (JDK 17以降推奨): `java -version`
- Node.js (Vue.js用): `node -v`
- Spring Boot CLI または Maven/Gradle: 依存関係の管理用

Codespacesのワークスペースにアクセスしなおす方法

ブラウザを閉じて開きなおし、再度Githubのリポジトリからワークスペースにアクセスするには、次の操作を行う必要があります。

- Githubのリポジトリ（今回は `flexible-attendance-app` ）をクリックする
- `<> Code` ボタンを押下して `codespaces` タブを開く
- タブ内に `on current branch` と記述された箇所があることを確認する
 - そこに `ubiquitous spork` といった名称が表示されている場合、それがここまでに編集してきたワークスペースである
 - 表示されている名称のワークスペースをクリックして、しばらく待機する

推奨されるディレクトリ構成

GitHub Codespaces（およびVS Code）でプロジェクト全体をスマートに管理するために、「**設定ファイル用のディレクトリ**」を追加しておくことをおすすめします。

Codespacesの設定を共通化するための `.devcontainer` を加えた構成がこちらです。

```
/flexible-attendance-app
├── .devcontainer (Codespacesの環境定義：VS Codeの拡張機能などを自動設定)
├── backend (Spring Bootプロジェクト)
├── frontend (Vue.jsプロジェクト)
└── docs (myapp_structures__修正版.md など)
```

なぜ `.devcontainer` があると良いのか？

Codespacesを一度閉じて再度立ち上げた際、JavaやVue.jsの開発に便利なVS Code拡張機能（Spring Boot Toolsなど）が最初からインストールされた状態にできるため、チーム開発や環境再現において非常に強力な武器になります。

環境構築のためのコマンド操作

ターミナルで以下のコマンドを順番に実行していきます。

```
# プロジェクトのルートディレクトリ（もし未作成なら）
```

```
mkdir -p flexible-attendance-app
```

```
cd flexible-attendance-app
```

```
# 3つの主要ディレクトリを作成
```

```
mkdir backend frontend docs
```

```
#（任意）設定用ディレクトリ
```

```
mkdir .devcontainer
```

バックエンドのプロジェクト生成

Spring Bootには「Spring Initializr」という便利な雛形作成ツールがありますが、Codespaces（VS Code）の**拡張機能**を使って作成するのが最も簡単です。

推奨される「依存関係（Library）」の選択

プロジェクト作成時に、以下のライブラリを含めるように設定してください。

1. **Spring Web**: RESTful API（Vue.jsとの通信）に必須。
2. **Spring Security**: パスワードハッシュ化（BCrypt）とアクセス制御（RBAC）用。
3. **Spring Data JPA**: データベース操作（MySQL/H2）を簡単にするため。
4. **MySQL Driver**: Aiven（MySQL）への接続用。
5. **H2 Database**: 開発初期の「とりあえず動かす」ための内蔵DBとして。
6. **Lombok**: Getter/Setterなどを自動生成してコードを綺麗に保つため。
7. **Validation**: サーバーサイドでのリクエストチェックに必須。

Spring Bootプロジェクト作成手順

拡張機能の確認

コマンドを有効にするために「Spring Boot拡張機能」がインストールされている必要があります。

- 画面左側の「Extensions（四角いアイコン）」をクリックし、検索欄に「Spring Boot Extension Pack」と入力してください。
- インストールされていなければ「Install」をクリックします。

コマンドパレットの起動とプロジェクト設定

1. 「Ctrl + Shift + P」 を押し、出た入力欄に `spring initializr` と入力します。
2. 「Spring Initializr: Create a Maven Project」 を選択します。（Gradleでも可能ですが、初学者には情報が多いMavenが一般的です）
3. **Spring Boot Version:** 特に出こだわりがなければ、最新の安定版（SNAPSHOTやM1と付いていないもの。例: `3.2.x` や `3.3.x` など）を選択します。
4. **Language:** `Java` を選択。
5. **Group Id:** `com.example` （任意ですが、そのままでもOKです）。
6. **Artifact Id:** `backend` と入力します。
7. **Packaging Type:** `Jar` を選択。
8. **Java Version:** `21` （Codespacesにインストールされているバージョンに合わせるのが無難です。25以降のバージョンが選択肢になれば最新のLTSである21で問題ありません）。

依存関係（Dependencies）の選択

次に、先ほど挙げたライブラリを検索して追加していきます。1つずつ入力して、候補に出てきたものを選択（チェック）してください。

- Spring Web
- Spring Security
- Spring Data JPA
- MySQL Driver
- H2 Database
- Lombok
- Validation

すべて選択し終わったら 「Enter」 を押します。

保存先の指定

最後に、どこにプロジェクトを展開するか聞かれます。

- 作成した `/flexible-attendance-app/backend` フォルダを指定してください。
- 「Generate into this folder」といったボタンを押すと、ファイルが展開されます。

実行後の確認

展開が完了すると、右下に「Open」というポップアップが出るか、ファイルエクスプローラーに `src` フォルダや `pom.xml` が表示されます。

これでバックエンドの「骨組み」が完成したことになります。

開発用データベース（H2）の有効化

Spring Bootプロジェクトが生成された直後は、設定（特にデータベース接続）が未完了のため、そのまま起動するとエラーが出ることがあります。

そのため、まずは「最小限の設定」を行い、起動を確認する手順を解説します。

まずは、外部のMySQL（Aiven）を使わなくても動くように、メモリ内で動作する **H2 Database** の設定を行います。

1. `backend/src/main/resources/application.properties` を開きます。
2. 以下の内容を貼り付けて保存してください。

```
# アプリケーション名
spring.application.name=flexible-attendance-app

# H2 Database 設定 (開発用の簡易DB)
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# H2コンソールをブラウザで確認できるようにする
spring.h2.console.enabled=true

# Spring Security のデフォルトログインを一時的に無効化 (動作確認を優先するため)
# ※後ほど、しっかりとしたセキュリティ設定を書く際に削除します
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

Spring Boot の起動

ターミナルを使って起動します。

1. ターミナルで `backend` ディレクトリに移動します。

```
cd /workspaces/flexible-attendance-app/backend
```

2. 以下のコマンドを実行します。

```
./mvnw spring-boot:run
```

起動確認

コマンドを実行すると、ターミナルにログが流れ始めます。最後に以下のようなメッセージが表示されれば成功です！

```
Started BackendApplication in X.XXX seconds (process running for X.XXX)
```

ポートの開放（GitHub Codespaces 特有の操作）

1. 起動に成功すると、画面右下に「A service is running on port 8080. Open in Browser」というポップアップが出ることがあります。
2. 「Open in Browser」をクリックするか、ターミナルの隣にある「Ports」タブを開き、8080 ポートの地球儀アイコンをクリックしてください。
3. ブラウザで新しいタブが開き、「Whitelabel Error Page (404)」と表示されれば**正常に動作しています**。サーバーが応答している証拠です。

詰まりやすいポイント

- **ポートが既に使用されている**: もしエラーで起動しない場合は、以前のプロセスが残っている可能性があります。その場合はターミナルで `Ctrl + C` を押して一度停止させてください。
- **mvnw の実行権限**: `permission denied` と出た場合は、`chmod +x mvnw` を実行して権限を与えてください。

ディレクトリ階層が間違っていた場合の対象法

`backend` ディレクトリの中にもう一つ同名のディレクトリを作成してしまい、その中に `.mvn` や `mvnw` といったファイルが格納されてしまっているとします。

つまり、ターミナル上で `cd /workspaces/flexible-attendance-app/backend` と入力してカレントディレクトリを切り替えた後、`ls -F` と入力した際に、`backend` としか表示されていない状態です。

この場合、ディレクトリ構成が `/workspaces/flexible-attendance-app/backend/backend` となってしまう為、末端の `backend` からファイル・フォルダを引き上げたうえで、この `backend` を削除する必要があります。

一つ上のディレクトリへのファイル・フォルダ移動

次のコマンドをターミナル上で実行します。

```
# 現在の場所を確認 (/workspaces/flexible-attendance-app/backend にいることを想定)
pwd

# 内側の backend の中身を、現在の場所 (.) に移動させる
# 「.*」は隠しファイルを指しますが、エラーを避けるため個別に指定します
mv backend/* .
mv backend/.mvn .
mv backend/mvnw .
mv backend/mvnw.cmd .
mv backend/pom.xml .
mv backend/.vscode .
mv backend/.gitattributes .
mv backend/.gitignore .

# もし `mv: cannot move 'backend/src' to './src': Directory not empty` のようなメッセージが出た場合は
# 既に一部のファイルが移動済みですので、そのまま次の削除ステップへ進んで大丈夫です
```

空になったディレクトリの削除

中身をすべて引き上げたら、以下のコマンドで古いディレクトリを消去します。

```
# 空になった内側のディレクトリを削除
rm -rf backend
```

改めて起動の最終確認

これで階層がスッキリしたはずです。改めて `ls -F` を実行し、直下に `src/` や `pom.xml` があることを確認したら、満を持して起動しましょう！

```
# 権限付与
chmod +x mvnw

# Spring Boot 起動
./mvnw spring-boot:run
```